

Architectural Paradigms in Neural Network Serialization: A Comprehensive Analysis of File Formats, Interoperability, and Pure Golang Integration Paths

Introduction to Neural Serialization Architectures

The deployment and operationalization of machine learning models have fundamentally transitioned from monolithic, framework-bound training scripts into a highly fragmented, edge-native, and interoperable ecosystem. Central to this structural evolution is the mechanism of neural network serialization—the highly specialized methodology by which high-dimensional tensor data, complex computation graphs, structural topologies, and localized hyperparameters are encoded, stored, and loaded across disparate hardware environments. The contemporary shift from heavily abstracted, runtime-dependent Python environments to high-performance, statically compiled edge-capable languages like Go necessitates a profound, byte-level understanding of these varied file formats.

This comprehensive research report conducts an exhaustive analysis of all major neural network file types, spanning from the memory-mapped operational efficiency of .safetensors to the intermediary, graph-oriented representations of the Open Neural Network Exchange (ONNX), and the edge-optimized flatbuffers of TensorFlow Lite (.tflite). It examines the exact versioning schemas, binary payload structures, and layer-level encoding strategies that dictate how varying artificial intelligence engines communicate, parse, and execute data. Furthermore, the analysis deeply investigates the limits of pure Golang (zero CGO) environments in supporting these formats. It focuses extensively on the Loom framework—a Deterministic Neural Virtual Machine (DNVM) capable of directly loading specific Hugging Face configurations into a three-dimensional volumetric grid. Finally, it outlines an architectural integration path to achieve universal format support entirely within native Go, addressing the critical limitations of implicit graph definitions and establishing a secure, cross-platform execution paradigm.

Exhaustive Taxonomy of Neural Network File Formats

The serialization of a neural network generally involves the persistent storage of two distinct but highly interrelated components: the topology (the computational graph defining layer types, connections, and hyperparameters) and the learned parameters (the multi-dimensional weights, biases, and quantization scales). Distinct formats prioritize different operational requirements, varying broadly between execution safety, zero-copy memory mapping, and cross-platform graph portability.

Format Designation	File Extension	Magic Number / Signature	Primary Origin / Execution Ecosystem	Graph Topology Encoded?
SafeTensors	.safetensors	N/A (JSON Header)	Hugging Face	No (Weights Only)
GGUF (v3)	.gguf	0x47 0x47 0x55 0x46	GGML / llama.cpp	Implicit via Metadata
ONNX	.onnx	Protobuf Signature	Open Neural Network Exchange	Yes (Explicit Nodes)
PyTorch Pickled	.pt, .pth	ZIP64 Signature	PyTorch Ecosystem	Yes (Python Object)
TensorFlow SavedModel	.pb / .pbtxt	Protobuf Signature	TensorFlow Core	Yes (MetaGraphDef)
TensorFlow Lite	.tflite	TFL3 (FlatBuffers)	TensorFlow Edge / Arm NN	Yes (FlatBuffer Schema)
Keras HDF5	.h5	\x89HDF\r\n\x1a\n	Keras / SciPy	Yes (JSON Attributes)

SafeTensors: The Memory-Mapped Weight Standard

Developed by the Hugging Face organization to address the critical security vulnerabilities and memory bottlenecks of Python's legacy pickle module, the .safetensors format has rapidly become the universal standard for weight-only serialization.¹ It algorithmically guarantees no arbitrary code execution upon loading and is architecturally designed for zero-copy memory mapping (mmap). This allows the host operating system to map the file directly into RAM or VRAM without requiring an intermediate allocation buffer on the CPU heap, bypassing a massive performance bottleneck during inference initialization.¹

The binary structure of the format is strictly declarative and relies on a simplistic, highly optimized byte layout. The file begins with an 8-byte little-endian unsigned 64-bit integer,

designated as N , which explicitly represents the exact byte size of the subsequent JSON header.¹ This integer is immediately followed by an N -byte UTF-8 encoded JSON string containing the tensor metadata. To maintain strict formatting, the header data must begin with the { character (0x7B) and may be padded with trailing whitespace (0x20) to align the subsequent raw tensor data to appropriate memory boundaries.¹

This JSON header functions as a comprehensive dictionary mapping tensor string names to their physical properties within the file. For example, a key such as "h.10.ln_1.weight" will map to a sub-dictionary defining the dtype (e.g., F32, F16, BF16, I8), the shape represented as an integer array, and the data_offsets, which is a two-element integer tuple `` indicating the exact byte offsets within the trailing byte-buffer where the tensor's raw binary data resides.¹

Furthermore, the header may contain a __metadata__ key utilized to store general model information, such as "format": "pt" to denote a PyTorch origin.³

Because SafeTensors is fundamentally a weight-storage mechanism, it does not define the mathematical computation graph. It relies entirely on the host AI engine to instantiate the correct neural architecture (such as a specific Transformer class) before systematically populating the programmatic nodes with the parsed weights.²

GGUF (GPT-Generated Unified Format): The Edge Inference Powerhouse

GGUF is a highly specialized binary format optimized for the rapid loading, memory mapping, and execution of quantized Large Language Models (LLMs) on commodity hardware, heavily utilized by the llama.cpp ecosystem and downstream executors like Ollama and LM Studio.⁵ It superseded the legacy GGML, GGMF, and GGJT formats by introducing a highly extensible key-value metadata store. This architectural shift solved the inherent brittleness of earlier formats that frequently broke backward compatibility whenever new hyperparameter dimensions were introduced by researchers.⁸

Currently stable at Version 3, the file structure of GGUF is composed of four sequential blocks that enforce strict memory alignment for execution speed. The format begins with a fixed-size header, starting with a 4-byte Magic Number (0x47 0x47 0x55 0x46, translating to GGUF in ASCII) which announces the file type to the parser.⁵ This is followed by a 32-bit unsigned integer denoting the format version (e.g., 3), an 8-byte integer representing the total tensor count, and an 8-byte integer defining the count of metadata key-value pairs.¹⁰

The second block is the Metadata Key-Value Store, a highly flexible sequence where each entry explicitly specifies the key length, the string key itself (such as "general.architecture" or "llama.context_length"), a value type enumeration, and the corresponding value.⁹ The third block comprises an array of Tensor Information structs. Each struct defines the tensor's name, a 32-bit integer for the number of dimensions (N), a sequence of N 64-bit integers defining the dimension sizes, a ggml_type enumeration dictating the exact quantization scheme, and a 64-bit offset pointing to the start of its raw data relative to the end of the tensor info array.⁹

Finally, the fourth block contains the consolidated tensor data. GGUF strictly enforces alignment (typically 32 or 64 bytes) dictated by the `general.alignment` metadata key. The parser mathematically calculates the required padding using the formula

$\text{align_offset}(\text{position}) - \text{position}$, padding the file with 0x00 bytes to ensure that every tensor begins at an optimal aligned memory address, which is critical for maximizing CPU cache hits during mmap loads.⁸

GGUF supports an extensive and highly mathematical array of integer quantization types, heavily utilizing K-quant schemes. In a legacy quantization block like Q4_0, a block consists of 32 weights. The data structure begins with a 2-byte FP16 scaling factor, followed by the quantized values packed into the lower and upper nibbles of the subsequent bytes. The operational formula $w = q \times \text{block_scale}$ is applied during the dequantization forward pass.⁶ The more advanced K-quants (e.g., Q4_K_M, Q5_K_S) improve upon this by employing a block-based scheme with separate scales and minimum values per super-block, significantly mitigating perplexity degradation at lower bit-widths.⁵ Experimental formats such as IQ2_XXS and 1.58-bit ternary quantization have also been seamlessly integrated into the `ggml_type` registry without requiring structural changes to the parser.⁵

ONNX (Open Neural Network Exchange): The Intermediate Representation

Unlike SafeTensors and GGUF, which are optimized for highly specific execution environments, ONNX serves as an Intermediate Representation (IR) specification built upon Protocol Buffers (Protobuf). It models both the exact mathematical computation graph and the tensor weights, ensuring that models can be ported across training frameworks like PyTorch and TensorFlow, and optimized execution runtimes like TensorRT and ONNX Runtime.¹⁵

ONNX utilizes two distinct and parallel versioning tracks that dictate graph compatibility. The IR Version is an atomic, monotonically increasing integer representing the structural format of the abstract graph protocol (e.g., IR version 9 or 10).¹⁸ Historically, up to IR version 6, ONNX only supported inference routing; version 7 structurally introduced training constructs, allowing gradient descent definitions to be passed between frameworks.¹⁵ Parallel to the IR version is the Opset Version (e.g., `ai.onnx` 19, 20, 21). Each operator set version represents a frozen snapshot of the available mathematical operators and their specific semantics.¹⁸ An engine attempting to load an ONNX file must mathematically support the specific opset; for instance, modern Large Language Models frequently require opsets that define fused SwiGLU or complex recurrent layers.¹⁸

ONNX Core Version	IR Version	Target Opset Version (ai.onnx)
1.14.1	9	19

1.15.0	9	20
1.16.0	10	21
1.16.2	10	21

The top-level construct within the serialized binary is `onnx.ModelProto`, which associates authoring metadata with an `onnx.GraphProto`.¹⁵ This internal graph contains the executable nodes (defined by an operation type like `MatMul` or `Add`), string-identified inputs, outputs, and initializers representing the constant weights.²⁰ However, because the underlying Protobuf serialization library strictly enforces a hard maximum file size limit of 2 Gigabytes, deep learning models exceeding this size must utilize an "external data" allocation schema. In this architecture, the primary `.onnx` protobuf file does not contain the binary weights; instead, it stores metadata strings detailing the location (a relative file path), the offset, and the length of the actual weights, effectively scattering the model across a directory structure.²¹

PyTorch Serialization (.pt,.pth) and Security Vectors

The native PyTorch file format, conventionally utilizing the `.pt` or `.pth` extensions, fundamentally represents a serialized dictionary of the model's `state_dict`, encapsulated within an uncompressed ZIP64 archive for all versions of PyTorch greater than 1.6.0.²²

The internal architecture of the archive is highly structural. It includes a pickled and serialized version of the primary object that was passed to `torch.save()`, a string identifying the `sys.byteorder` (typically "little"), a version number, and a `data/` directory housing individual files for each tensor storage object.²² PyTorch conceptually separates "views" from "storage." Multiple tensors in a model may logically point to the exact same underlying binary memory block with different strides and shapes, allowing for highly efficient slicing.²³ When PyTorch serializes these tensors, it saves the shared storage objects and the tensor metadata separately, linking them during the `torch.load()` reconstruction phase.²³

This serialization methodology, however, introduces severe structural vulnerabilities. Because the format relies fundamentally on Python's pickle utility, deserialization relies on an opcode-based virtual machine capable of executing arbitrary Python instructions embedded within the stream.²³ By injecting malicious opcodes such as `REDUCE`, `BUILD`, or `GLOBAL` into the pickle stream, an attacker can reference dangerous modules like `os` or `subprocess`.²⁵ This leads directly to Common Weakness Enumeration (CWE) 345 vulnerabilities, notably cataloged under CVE-2025-1945, where maliciously crafted `.pth` files can bypass ZIP file header validation to spawn hidden background processes, execute shell commands, or deploy ransomware immediately upon invoking the loading function.²⁴ To mitigate this systemic architectural flaw, PyTorch introduced the `weights_only=True` parameter, which forcibly restricts the unpickler to a strict allowlist of mathematical classes, though the foundational risk of utilizing an

execution-based format for static data transport remains.²³

TensorFlow Ecosystem: SavedModel and TFLite FlatBuffers

TensorFlow segments its serialization strategy based on the intended deployment hardware. Server-side deployments utilize the SavedModel directory format, while mobile and edge environments utilize the highly optimized .tflite format.²⁷

The SavedModel format acts as a unified, directory-based serving configuration containing both execution logic and parameter states. The root directory contains either saved_model.pb or saved_model.pbtxt, which are Protobuf files holding the MetaGraphDef (defining the structural training and inference logic) and the SignatureDef (specifying the exact input and output interface tensors for model serving APIs like Triton Inference Server).²⁹ Alongside the Protobuf files is a variables/ directory containing the weights, which are often split across multiple shards (e.g., variables.data-00000-of-00001) and indexed by a mapping file (variables.index).²⁹

For edge environments, TensorFlow collapses this highly modular graph into a single .tflite file utilizing Google's FlatBuffers serialization library.²⁸ FlatBuffers fundamentally differs from Protobuf; it allows for direct memory access without the computational overhead of parsing, unpacking, or allocating secondary representations in heap memory.²⁸ The structural schema is explicitly defined via files like schema_v3.fbs, which the flatc compiler uses to generate language-specific accessor classes.³³ By avoiding parsing steps, a mobile processor can execute a .tflite model directly from the memory-mapped file, conserving battery life and minimizing latency.²⁸

Keras HDF5 (.h5)

The Hierarchical Data Format version 5 (HDF5) represents the legacy standard for Keras, persisting both the entire model architecture and the multi-dimensional weight arrays within a single file.³⁶ HDF5 functions conceptually as a virtual file system contained within a binary file, offering robust support for massive datasets and hierarchical grouping.³⁷ In Keras .h5 files, the layer architecture and compilation attributes are serialized as JSON strings and stored as HDF5 group attributes, while the weights themselves are saved as high-performance, contiguous numerical arrays within the nested group hierarchy.³⁷ While highly robust, the complexity of the HDF5 C-library has largely led the modern edge industry to favor the simplicity of .safetensors for isolated weight storage, particularly outside the Python ecosystem.³⁷

Engine Interoperability and the Format Translation Matrix

Understanding exactly what AI engines communicate with each other via file formatting reveals a highly interdependent, yet often computationally lossy, ecosystem. Engines rarely speak the exact same language natively without intermediate conversion steps that often alter the mathematical fidelity of the model.

Source Engine / Format	Intermediate Translation Path	Final Target Runtime	Fidelity Loss Vector
PyTorch (.pt)	ONNX Converter (Opset matching)	TensorRT / ONNX Runtime	Operation Shift; unsupported nodes must be decomposed.
PyTorch / SafeTensors	convert_hf_to_gguf.py	llama.cpp / Ollama	Quantization precision loss (PTQ rounding).
TensorFlow SavedModel	TFLite Converter / FlatBuffer Optimizer	Arm NN / Edge TPU	Operator pruning; dynamic range quantization.
Keras .h5	TensorFlow SavedModel Exporter	TensorFlow Serving	Structural extraction; minimal mathematical loss.
Hugging Face SafeTensors	Direct Mmap Ingestion	Loom M-POLY-VTD Engine	None (If engine natively supports the topology).

The translation of models frequently results in "Operation Shifts." For example, transforming a PyTorch network to ONNX requires that the specific PyTorch mathematical operations perfectly align with the target ONNX opset.¹⁶ If a custom PyTorch operation lacks an ONNX equivalent, the converter must either decompose it into primitive mathematical nodes or fail entirely.¹⁷

When quantization is introduced, engines frequently break strict mathematical compatibility. PyTorch implements Quantization-Aware Training (QAT) via "fake quantization" nodes injected into the graph, which simulate precision loss during the training forward pass to allow the optimizer to adjust. Conversely, GGUF and llama.cpp rely heavily on Post-Training Quantization (PTQ), encoding strict numerical degradation explicitly in the data type `ggml_type` and requiring specialized dequantization kernels during the inference pass.⁵

The Loom Framework and the SafeTensors Structural Constraint

The highest expression of a pure Go neural execution environment identified in this research is the Loom framework (currently at v0.79 Bedrock). Authored by the OpenFluke lab, Loom is engineered as a Deterministic Neural Virtual Machine (DNVM), self-characterized as the "SQLite of AI".⁴⁰

Unlike frameworks that act as simple language wrappers around massive C++ backends (like libtorch or libtensorflow), Loom is a ground-up neural engine built entirely without CGO, natively supporting execution across Windows, Linux, macOS, iOS, Android, and browsers via WebAssembly (WASM).⁴³ Loom attempts to bypass traditional model conversion constraints by ingesting Hugging Face .safetensors files directly, sidestepping the lossy conversion pipelines required to reach ONNX or TFLite.⁴⁵

The M-POLY-VTD Architecture

Loom abandons the linear, stack-based sequence of traditional frameworks (such as nn.Sequential in PyTorch). Instead, it implements a 3D Volumetric Coordinate System.⁴² The neural network is modeled mathematically as a four-dimensional array of VolumetricLayer structs, indexed spatially by (Depth, Row, Col, LayerIndex).⁴⁷

This architecture, termed the Multi-Numerical POLYmorphic Volumetric Tiled-tensor Dispatcher (M-POLY-VTD), operates upon several foundational pillars:

1. **Multi-Numerical Execution:** Loom natively computes forward and backward passes across 21 distinct numerical data types (DTypes), stretching from high-precision float64 all the way down to int4, ternary, and experimental 1-bit binary arrays.⁴² This allows it to natively map advanced quantization formats like Microsoft's BitNet b1.58 directly to CPU logic.⁴²
2. **Volumetric Tensor Dispatch:** Data flows through a 3D "Step Mesh Engine" where spatial links simulate biological neuronal firing. Parallel layers generate sub-activation trees stored within a Nested property of the Tensor struct, allowing branches to execute asynchronously and independently before merging.⁴²
3. **Neural Target Propagation (Tween):** Rather than standard algorithmic backpropagation, Loom utilizes localized target propagation. Each layer receives a distinct target, allowing for gap-based Hebbian learning driven by the formula $\Delta W = \eta \cdot \text{input} \cdot (\text{target} - \text{actual})$.⁴² This bypasses the need for strictly differentiable functions, permitting optimization over 1-bit binary layers where classical gradients would vanish.⁴⁸
4. **Absolute Determinism:** Loom promises bitwise-identical execution. A calculation run on a mobile CPU will produce the exact same tensor outputs (Mean Absolute Error < 1e-8) as an execution pushed through WebGPU in a browser.⁴⁵ This is accomplished through highly explicit floating-point definitions programmed in pure Go, bypassing the nondeterministic reductions typical of varying CUDA libraries.⁴⁵

The SafeTensors Topology Limitation

Despite its profound capabilities, the original query astutely notes a critical constraint: Loom

can support .safetensors, *but only on certain models*.

The technical reasoning for this limitation is rooted in the structure of the file formats themselves. As previously established, SafeTensors is strictly a dictionary of flat byte arrays; it contains only the weights.¹ It does *not* contain the computational graph or the architectural topology of the model.²

Therefore, for Loom to load a SafeTensors file, it must execute Go APIs such as `nn.LoadTransformerFromSafetensors("models/SmolLM2-360M")`.⁴⁶ This API function relies on the fact that the specific architectural topology of SmolLM2, LLaMA, or Qwen has already been hardcoded into Loom's Go source code. Loom's engine contains pre-programmed 3D volumetric jump tables that establish the exact sequence of Multi-Head Attention (MHA), Rotary Position Embeddings (RoPE), Grouped-Query Attention (GQA), SwiGLU activations, and RMSNorm layers required by these specific models.⁴⁴ The weights from the file are merely poured into this pre-existing structural mold.

If a developer downloads a SafeTensors file for a highly novel, experimental architecture that Loom has not yet explicitly programmed into its Go source tree, the load sequence will irreparably fail. The engine cannot guess the topological arrangement of the network purely from the tensor weights. This highlights the inherent trade-off of weight-only formats versus graph-aware formats like ONNX.

The Universal Integration Path in Pure Golang

The fundamental requirement of the query dictates an exploration into building an integration path to support *everything* conceivable in pure Golang, and pushing the limits of versioning and file-type formatting without relying on CGO wrappers. Bridging the gap from Loom's current state to universal format support (GGUF, ONNX, TFLite, PyTorch, HDF5) requires the establishment of a unified, four-phase parsing, extraction, and graph-construction pipeline entirely in native Go.

Phase 1: Universal Abstract Syntax Tree (AST) for Neural Topologies

To resolve the limitation of implicit graph definitions found in SafeTensors, the pure Go integration path must begin with an intermediate topological abstraction. The engine must compile varying file formats into a unified Go struct hierarchy representing a universal computation graph.

- **Ingesting ONNX Protobuf Natively:** To read ONNX files without CGO, the pipeline must utilize packages such as `github.com/owulveryck/onnx-go` (recently maintained by Orama).⁵¹ By generating Go structures directly from the ONNX Protobuf schemas, the pipeline can read the `onnx.ModelProto` and iterate over the explicit computation nodes.²⁰ The Go parser must handle the varying IR versions (9, 10) and translate the Opset definitions (19, 20, 21) into Go interface implementations.¹⁸
- **Ingesting TFLite FlatBuffers Natively:** Similarly, parsing TFLite files is achieved by utilizing the Go API generated directly by the flatc compiler (e.g., `github.com/mattrobertson/go-tflite` or `nbortolotti/tflitego`).⁵³ The engine reads the `schema_v3.fbs`

file, interpreting the schema to provide direct memory-slice access to the FlatBuffers byte array without allocation overhead.²⁸

- **Ingesting Keras HDF5 Natively:** Reading .h5 files natively in Go was historically impossible without CGO bindings to the massive HDF5 C-library. However, this barrier has been shattered by the advent of packages like github.com/scigolib/hdf5 (v0.11.4-beta) and [scigolib/matlab](https://github.com/scigolib/matlab). These packages represent a modern, pure Go implementation of the HDF5 v2.0.0 format specification.³⁸ They allow the Go engine to parse the superblock, navigate the B-Tree hierarchy, read chunked datasets with GZIP decompression, and extract the JSON-encoded Keras architecture directly into the AST.³⁸
- **Heuristic Graph Assembly for SafeTensors/GGUF:** Because SafeTensors and GGUF lack explicit graph nodes, the integration path must utilize a heuristic graph assembler. This assembler parses the key strings (e.g., "blk.12.ffn_down.weight") and the GGUF metadata key-value store, inferring the transformer topology through pattern matching and automatically constructing the AST.⁹

Phase 2: Memory-Safe Buffer Abstraction

A critical friction point in Go involves managing the lifecycle of multi-gigabyte matrix arrays. If Go's automated Garbage Collector (GC) is forced to scan and manage a 10GB tensor payload allocated on the heap, the resulting stop-the-world GC pauses will render inference latency unacceptable. The universal Go path must aggressively rely on the unsafe package and memory mapping to bypass the GC.

- **Handling SafeTensors and GGUF:** The Go integration maps the file into memory using `syscall.Mmap`. Once the metadata header is parsed and the exact `data_offsets` are obtained, the integration uses `unsafe.Pointer` to cast the raw byte slice directly into multidimensional pointers (e.g., `*[...][...]float32`) representing the weights.¹ For GGUF files, because they group quantizations in rigid memory blocks (e.g., 32 parameters mapped to a 2-byte FP16 scale), Go must implement a highly optimized bitwise unpacker to dequantize the values natively into the pipeline, observing the strict 32/64-byte alignment boundaries defined by `general.alignment`.⁸
- **Handling PyTorch (.pt):** Loading PyTorch files natively without executing malicious Python opcodes requires an isolated unpickler. The github.com/nlpodyssey/gopickle library provides this exact functionality.⁵⁹ Simulating Python's Pickle protocols (versions 0 through 5), `gopickle` strictly reads the structural metadata and extracts PyTorch's primitive Storage classes as raw byte sequences without instantiating an execution VM.⁵⁹ Once the ZIP64 archive is parsed and the uncompressed raw bytes are located, the integration streams the bytes directly into the pre-allocated Go tensor buffers.²² This mechanism entirely neutralizes the CVE-2025-1945 attack vector.²⁵

Phase 3: Volumetric Graph Mapping and Topological Signatures

Once the Universal AST is constructed and the weights are mapped to memory-safe buffers, the varying graphs must be translated into the execution engine. In an advanced 3D paradigm

like Loom's M-POLY-VTD, standard sequential operations (like a TensorFlow Dense layer followed by a Softmax) must be assigned distinct coordinates in the volumetric mesh: (Depth, Row, Col, LayerIndex).⁴⁷

A translation layer must programmatically walk the generated AST and assign a spatial address to every node. An ONNX MatMul node, followed by an Add node²⁰, maps to consecutive addresses. If branching occurs, such as in Mixture of Experts (MoE) architectures or dense ResNet residual connections, the integration pipeline leverages the Nested property of the Tensor type to establish remote spatial links, allowing the gradients to propagate backward through the parallel branches correctly.⁴⁴ Furthermore, Loom's DNA Engine can extract the topological "signature" of this newly imported graph, establishing a highly accurate structural fingerprint that permits evolutionary recombination and verification against architectural shifts.⁴¹

Phase 4: Hardware Limits, WebGPU, and Edge Execution

How far can a pure Golang runtime scale this universal format pipeline? The theoretical and practical limits are bounded by two distinct factors: the execution speed of raw mathematical loops on the CPU versus hardware-accelerated kernels, and the availability of unified graphics APIs.

1. **CPU Execution and Assembly Limits:** Pure Go mathematical operations lacking explicit SIMD (AVX2/AVX-512) compiler intrinsics will naturally underperform against heavily optimized C++ backends like llama.cpp or OpenBLAS. Frameworks like Loom mitigate this by utilizing direct Go Assembly (poly/asm) for highly repetitive dense forward passes.⁴⁰ The true limit here is the developer burden of maintaining highly specific ASM implementations across the diverging instruction sets of x86_64, ARM64, and Apple Silicon architectures.⁴⁰
2. **GPU Backend Limits (Zero CGO Unification):** The strict architectural prohibition on CGO precludes the use of standard NVIDIA CUDA, AMD ROCm, or Intel oneAPI libraries natively within Go.⁴³ However, this limitation has been vastly extended by the rapid maturation of WebGPU. Loom successfully circumvents the CGO limit by bridging Go directly into native WebGPU specifications.⁴⁰ By compiling the neural network layers directly into the WebGPU Shading Language (WGSL) and executing them on the graphics card without C-ABI middleware, pure Go can achieve parity with lower-level system languages on consumer hardware, even when deployed to a web browser via highly constrained WASM bundles.⁴² The limitation currently rests in optimizing the in-memory SafeTensors transfers to the WebGPU command buffers, a high-priority frontier in current Go AI research.⁴⁴

Conclusion

The neural network serialization landscape is heavily stratified and deeply complex. ONNX remains mathematically necessary for framework-agnostic computation graphs; GGUF excels exponentially at quantized LLM deployment for fast edge inference; PyTorch .pt dictates initial

training regimes despite its severe pickle-based execution vulnerabilities; and Hugging Face's .safetensors has effectively solved the security and zero-copy latency demands for weight distribution, though it severely limits interoperability by abandoning explicit architectural mapping.

To support all file formats within a pure Golang environment with absolutely zero CGO dependencies—a requisite for maximum portability across Edge, WASM, and enterprise systems—requires an architecture capable of extreme morphological flexibility. The Loom framework demonstrates that this is not only possible but performant, through its multi-numerical DType support, WebGPU unification, and biological 3D step mesh design. By strategically extending such a Deterministic Neural Virtual Machine with a universal intermediate topological parser—one capable of interpreting Protobuf (ONNX), FlatBuffers (TFLite), Pickle structs via safe extraction (PyTorch), and B-Tree hierarchies (HDF5)—the Go language can serve as the ultimate universal machine learning integration path. Bypassing the historical limitations of the Python ecosystem and the heavy footprint of C++ compilation, a properly memory-aligned, purely Go-based engine establishes a robust paradigm for secure, offline, and bit-exact neural deployments across any conceivable hardware substrate.

Works cited

1. GitHub - safetensors/safetensors: Simple, safe way to store and distribute tensors, accessed May 29, 2026, <https://github.com/safetensors/safetensors>
2. Safetensors - Hugging Face, accessed May 29, 2026, <https://huggingface.co/docs/safetensors/index>
3. Metadata Parsing - Hugging Face, accessed May 29, 2026, https://huggingface.co/docs/safetensors/metadata_parsing
4. Safetensors File Format - CRAN, accessed May 29, 2026, <https://cran.r-project.org/web/packages/safetensors/safetensors.pdf>
5. GGUF - Wikipedia, accessed May 29, 2026, <https://en.wikipedia.org/wiki/GGUF>
6. GGUF · Hugging Face, accessed May 29, 2026, <https://huggingface.co/docs/hub/gguf>
7. GGUF · Hugging Face, accessed May 29, 2026, <https://huggingface.co/docs/hub/en/gguf>
8. ggml/docs/gguf.md at master · ggml-org/ggml - GitHub, accessed May 29, 2026, <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
9. GGUF in details. After Training phase, the models based... | by Charles Vissol - Medium, accessed May 29, 2026, <https://medium.com/@charles.vissol/gguf-in-details-8a9953ac7883>
10. LLM GGUF Guide: File Format, Structure, and How It Works - ApX Machine Learning, accessed May 29, 2026, <https://apxml.com/posts/gguf-explained-llm-file-format>
11. GGUF - GitHub, accessed May 29, 2026, <https://raw.githubusercontent.com/ggml-org/ggml/master/docs/gguf.md>
12. A Short Guide to the GGUF Format - Gianluca Guida's personal page., accessed May 29, 2026, http://tlbflush.org/post/2025_02_17_gguf_weekend/

13. Need help to understand q4_0, q4_1, q4_2, q4_3 quantization · ggml-org llama.cpp · Discussion #1121 - GitHub, accessed May 29, 2026, <https://github.com/ggml-org/llama.cpp/discussions/1121>
14. Overview of GGUF quantization methods : r/LocalLLaMA - Reddit, accessed May 29, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1ba55rj/overview_of_gguf_quantization_methods/
15. onnx/docs/IR.md at main · onnx/onnx - GitHub, accessed May 29, 2026, <https://github.com/onnx/onnx/blob/main/docs/IR.md>
16. Onnx format, how it works? - Stack Overflow, accessed May 29, 2026, <https://stackoverflow.com/questions/60475686/onnx-format-how-it-works>
17. Internals - FINN documentation, accessed May 29, 2026, <https://finn.readthedocs.io/en/latest/internals.html>
18. onnx/docs/Versioning.md at main - GitHub, accessed May 29, 2026, <https://github.com/onnx/onnx/blob/main/docs/Versioning.md>
19. Open Neural Network Exchange Intermediate Representation (ONNX IR) Specification, accessed May 29, 2026, <https://onnx.ai/onnx/repo-docs/IR.html>
20. ONNX Concepts - ONNX 1.22.0 documentation, accessed May 29, 2026, <https://onnx.ai/onnx/intro/concepts.html>
21. Working with Large Models | onnxruntime, accessed May 29, 2026, <https://onnxruntime.ai/docs/tutorials/web/large-models.html>
22. PyTorch Serialized File Format - Library of Congress, accessed May 29, 2026, <https://www.loc.gov/preservation/digital/formats/fdd/fdd000644.shtml>
23. Serialization semantics — PyTorch 2.12 documentation, accessed May 29, 2026, <https://docs.pytorch.org/docs/stable/notes/serialization.html>
24. Python Pickle Poisoning and Backdooring Pth Files | Snyk, accessed May 29, 2026, <https://snyk.io/articles/python-pickle-poisoning-and-backdooring-pth-files/>
25. CVE-2025-1945: Mmaitre314 Picklescan RCE Vulnerability - SentinelOne, accessed May 29, 2026, <https://www.sentinelone.com/vulnerability-database/cve-2025-1945/>
26. Zero-Trust Artificial Intelligence Model Security Based on Moving Target Defense and Content Disarm and Reconstruction - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2503.01758v1>
27. Introduction to modules, layers, and models | TensorFlow Core, accessed May 29, 2026, https://www.tensorflow.org/guide/intro_to_modules
28. FlatBuffers Docs, accessed May 29, 2026, <https://flatbuffers.dev/>
29. Platform For AI:FAQ about TensorFlow - Alibaba Cloud, accessed May 29, 2026, <https://www.alibabacloud.com/help/en/pai/faq-about-tensorflow>
30. tutorials/Quick_Deploy/TensorFlow/README.md at main · triton-inference-server/tutorials - GitHub, accessed May 29, 2026, https://github.com/triton-inference-server/tutorials/blob/main/Quick_Deploy/TensorFlow/README.md
31. Exploration of Tensorflow SavedModel | by Jimmy (xiao) Shen - Medium, accessed May 29, 2026, <https://jimmy-shen.medium.com/exploration-of-tensorflow-savedmodel-cf6e585>

[9cc5c](#)

32. How TensorFlow Lite helps you from prototype to product, accessed May 29, 2026,
<https://blog.tensorflow.org/2020/04/how-tensorflow-lite-helps-you-from-prototype-to-product.html>
33. TensorFlow Lite Flatbuffer Manipulation Example - Colab - Google, accessed May 29, 2026,
https://colab.research.google.com/drive/11R9hd_OzvW0y3mesitNtF8dCjf27xP9T?usp=sharing
34. Studying Flatbuffers to play with TFLite models - Ricardo's Place, accessed May 29, 2026,
https://ricardodeazambuja.com/deep_learning/2021/05/23/tflite-flatbuffers/
35. Deployment pipelines and enhanced modularity in Kenning - Antmicro, accessed May 29, 2026,
<https://antmicro.com/blog/2022/09/deployment-pipelines-and-enhanced-modularity-in-kenning>
36. Saving and Loading Keras Models in .h5 Format: A Step-by-Step Guide - Medium, accessed May 29, 2026,
<https://medium.com/@stacymacbrains/saving-and-loading-keras-models-in-h5-format-a-step-by-step-guide-b81f4b7ec8e2>
37. How to Save and Load Your Keras Deep Learning Model - MachineLearningMastery.com, accessed May 29, 2026,
<https://machinelearningmastery.com/save-load-keras-deep-learning-models/>
38. Pure Go library for reading and writing MATLAB .mat files (v5-v7.3+). No CGo, no external dependencies. Full support for numeric types, complex numbers, and multi-dimensional arrays. Cross-platform (Windows/Linux/macOS). Part of SciGoLib ecosystem. · GitHub, accessed May 29, 2026,
<https://github.com/scigolib/matlab>
39. weights and biases from hdf5 file - python - Stack Overflow, accessed May 29, 2026,
<https://stackoverflow.com/questions/55155759/weights-and-biases-from-hdf5-file>
40. OpenFluke — Sovereign AI on Your Hardware (Golang AI Engine), accessed May 29, 2026, <https://openfluke.com/>
41. loom - Layered Omni-architecture Openfluke Machine - GitHub, accessed May 29, 2026, <https://github.com/openfluke/loom>
42. Loom — Golang AI Engine, Portable & Zero CGO (v0.79) - OpenFluke, accessed May 29, 2026, <https://openfluke.com/loom>
43. I built a neural runtime in pure Go (no CGO, no PyTorch). It runs real-time learning in the browser via WASM. : r/golang - Reddit, accessed May 29, 2026,
https://www.reddit.com/r/golang/comments/1pphahe/i_built_a_neural_runtime_in_pure_go_no_cgo_no/
44. High-Performance GPU Compute in Go: Releasing Loom v0.0.8 (Native WebGPU & LLM Primitives) : r/golang - Reddit, accessed May 29, 2026,
https://www.reddit.com/r/golang/comments/1qh1677/highperformance_gpu_com

- [pute_in_go_releasing_loom/](#)
45. [P] LOOM: Universal ML runtime with cross-platform determinism (Go-based, loads HuggingFace directly) : r/learnmachinelearning - Reddit, accessed May 29, 2026, https://www.reddit.com/r/learnmachinelearning/comments/1ouzpqz/p_loom_universal_ml_runtime_with_crossplatform/
 46. LOOM: The Universal AI Runtime That Works Everywhere (And Why That Matters) - Medium, accessed May 29, 2026, <https://medium.com/@planetbridging/loom-the-universal-ai-runtime-that-works-everywhere-and-why-that-matters-54de5e7ec182>
 47. M-POLY-VTD: Architecture Overview — Loom Docs - OpenFluke, accessed May 29, 2026, <https://openfluke.com/docs/overview>
 48. Loom M-POLY-VTD — Golang AI Architecture Deep Research - OpenFluke, accessed May 29, 2026, <https://openfluke.com/loom/research>
 49. [D] Self-Promotion Thread : r/MachineLearning - Reddit, accessed May 29, 2026, https://www.reddit.com/r/MachineLearning/comments/1om5smw/d_selfpromotion_thread/
 50. Loom Docs — Golang AI Engine Reference - OpenFluke, accessed May 29, 2026, <https://openfluke.com/docs>
 51. examples/ directory - github.com/owulveryck/onnx-go/examples - Go Packages, accessed May 29, 2026, <https://pkg.go.dev/github.com/owulveryck/onnx-go/examples>
 52. onnx-go gives the ability to import a pre-trained neural network within Go without being linked to a framework or library. - GitHub, accessed May 29, 2026, <https://github.com/oramasearch/onnx-go>
 53. mattn/go-tflite: Go binding for TensorFlow Lite - GitHub, accessed May 29, 2026, <https://github.com/mattn/go-tflite>
 54. tflite package - github.com/nbortolotti/tflitego - Go Packages, accessed May 29, 2026, <https://pkg.go.dev/github.com/nbortolotti/tflitego>
 55. hdf5 package - github.com/scigolib/hdf5 - Go Packages, accessed May 29, 2026, <https://pkg.go.dev/github.com/scigolib/hdf5>
 56. Pure Go HDF5 Library - Production-Ready Write Support (v0.11.4-beta) - HDF Forum, accessed May 29, 2026, <https://forum.hdfgroup.org/t/pure-go-hdf5-library-production-ready-write-support-v0-11-4-beta/13572>
 57. Scientific file formats: HDF5, FITS, NetCDF? - Google Groups, accessed May 29, 2026, <https://groups.google.com/g/golang-nuts/c/vJOEcmjclWw>
 58. safetensors package - github.com/nlpodyssey/safetensors - Go Packages, accessed May 29, 2026, <https://pkg.go.dev/github.com/nlpodyssey/safetensors>
 59. nlpodyssey/gopickle: Go library for loading Python's data ... - GitHub, accessed May 29, 2026, <https://github.com/nlpodyssey/gopickle>
 60. nlpodyssey/verbaflow: Neural Language Model for Go - GitHub, accessed May 29, 2026, <https://github.com/nlpodyssey/verbaflow>
 61. @openfluke/welvet - npm, accessed May 29, 2026, <https://www.npmjs.com/package/@openfluke/welvet>